

PySpark Interview Questions & Answers — Complete Guide

42 practical questions covering PySpark fundamentals, optimization, salting, Delta Lake, and Spark architecture

Interview revision guide: Focus on understanding what each concept does, when to use it, and how it affects Spark performance.

1. PySpark Fundamentals

1. What is PySpark?

PySpark is the Python API for Apache Spark. It allows developers to process large datasets in a distributed manner using Spark's execution engine.

2. What is SparkSession?

SparkSession is the main entry point for working with DataFrames, SQL, and many Spark features in modern PySpark applications.

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("Demo").getOrCreate()
```

3. What is the difference between RDD and DataFrame?

RDD is a low-level distributed collection of objects. A DataFrame is a distributed table with named columns and a schema. DataFrames generally provide better optimization through Spark SQL's Catalyst optimizer and are easier to use for structured data.

4. What is lazy evaluation in Spark?

Spark does not immediately execute transformations. It builds a logical execution plan and executes it when an action is called. This allows Spark to optimize the plan before processing the data.

```
df = df.filter(df.salary > 50000) # transformation
df.show() # action
```

5. What are transformations and actions?

Transformations create a new dataset from an existing one, such as `select()`, `filter()`, `map()`, and `groupBy()`. Actions trigger execution and return a result or write data, such as `show()`, `count()`, `collect()`, and `write()`.

6. What is the difference between narrow and wide transformations?

A narrow transformation can process data without a shuffle across partitions, for example `filter()` and `select()`. A wide transformation requires data movement between partitions, for example `groupBy()`, `join()`, and `distinct()`. Wide transformations are usually more expensive because they can cause shuffling.

7. What is `repartition()` vs `coalesce()`?

`repartition()` can increase or decrease the number of partitions and generally causes a full shuffle. `coalesce()` is mainly used to reduce partitions and can avoid a full shuffle in common cases. Use them carefully because partitioning affects performance.

```
df = df.repartition(10)
df = df.coalesce(4)
```

8. What is `cache()` vs `persist()`?

`cache()` stores a dataset using Spark's default persistence level. `persist()` lets you explicitly choose a storage level such as `MEMORY_AND_DISK`. Both can avoid recomputation when a dataset is reused.

```
df.cache()

from pyspark import StorageLevel
df.persist(StorageLevel.MEMORY_AND_DISK)

df.unpersist()
```

9. What is a broadcast join?

A broadcast join sends a small table to each executor so Spark can join it with a large table without shuffling the large table. It can be useful when one side of the join is small enough to broadcast safely.

```
from pyspark.sql.functions import broadcast
result = large_df.join(broadcast(small_df), "customer_id")
```

10. What is a window function in PySpark?

A window function performs calculations across related rows while keeping the original rows. Common examples include `row_number()`, `rank()`, `lag()`, `lead()`, and running totals.

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

w = Window.partitionBy("customer_id").orderBy("order_date")
df.withColumn("rn", row_number().over(w))
```

11. How do you remove duplicate records?

For complete-row duplicates, use `dropDuplicates()`. You can also provide specific columns when duplicates should be identified based on selected keys.

```
df = df.dropDuplicates()
df = df.dropDuplicates(["customer_id", "order_date"])
```

12. How do you handle null values?

PySpark provides functions such as `dropna()`, `fillna()`, and `when()/otherwise()`. The correct method depends on whether the null should be removed, replaced, or handled conditionally.

```
df = df.dropna()
df = df.fillna({"salary": 0})
```

13. How do you read and write CSV data?

Use `spark.read.csv()` to load CSV data and `DataFrame.write.csv()` to save it. In production pipelines, explicitly defining schemas is often preferable to relying on inference.

```
df = spark.read.option("header", True).option("inferSchema", True).csv("data.csv")
df.write.mode("overwrite").option("header", True).csv("output/")
```

14. Why is `collect()` risky for large datasets?

`collect()` brings all records from the executors to the driver. If the dataset is large, this can consume driver memory and cause an out-of-memory error. Prefer distributed operations such as `groupBy()`, `write()`, or carefully limited `show()/take()` when appropriate.

15. How can you improve PySpark job performance?

Common techniques include selecting only required columns, filtering early, using appropriate partitioning, avoiding unnecessary shuffles, using broadcast joins for genuinely small tables, handling data skew, caching only reused datasets, and using Spark's execution plan tools to identify bottlenecks.

```
df.select("customer_id", "amount").filter("amount > 1000").explain(True)
```

2. Optimization & Performance

16. What are the main PySpark optimization techniques?

Common techniques include filtering early, selecting only required columns, avoiding unnecessary shuffles, choosing an appropriate partition count, using broadcast joins for genuinely small tables, handling data skew, caching only reused data, using efficient file formats such as Parquet/Delta, and inspecting the execution plan with `explain()`.

17. What is predicate pushdown?

Predicate pushdown means applying filters as close to the data source as possible so Spark can avoid reading unnecessary rows. Columnar formats such as Parquet can benefit significantly from this optimization.

```
df = spark.read.parquet("sales/").filter("amount > 1000")
```

18. Why should you select only required columns?

Reading unnecessary columns increases I/O, network transfer, memory usage, and processing time. Selecting only the columns required by the transformation or output can make a Spark job more efficient.

```
df = df.select("customer_id", "amount", "order_date")
```

19. What is data skew and how do you handle it?

Data skew occurs when some partitions contain much more data than others. One or a few tasks can then take much longer than the rest. Techniques include salting, better partitioning, pre-aggregation, broadcast joins where appropriate, and adaptive query execution features.

20. What is Adaptive Query Execution (AQE)?

AQE allows Spark to adjust parts of a query plan at runtime using statistics collected during execution. Depending on the Spark version and configuration, AQE can help with issues such as skewed joins, partition coalescing, and join strategy selection.

```
spark.conf.set("spark.sql.adaptive.enabled", "true")
```

21. How do you identify a performance bottleneck in a Spark job?

Start by checking the Spark UI for stages, task duration, shuffle read/write, input size, and failed or slow tasks. Use `explain()` to inspect the physical plan and look for expensive shuffles, scans, joins, or unexpected operations.

```
df.explain("formatted")
```

22. Why is Parquet often preferred over CSV for analytics?

Parquet is a columnar format that stores schema and column-level data efficiently. It can reduce I/O through column pruning and predicate pushdown. CSV is text-based and generally requires more parsing and storage for the same analytical data.

23. What is partition pruning?

Partition pruning allows Spark to read only the relevant partitions when a query filters on a partition column. This can substantially reduce the amount of data scanned.

3. Data Skew & Salting

24. What is the salting technique in PySpark?

Salting is a technique used to distribute records with a heavily repeated key across multiple partitions. A small salt value is added to the skewed key so that one hot key does not concentrate all processing in a single partition.

```
from pyspark.sql import functions as F

salted = df.withColumn("salt", F.floor(F.rand() * 10))
salted = salted.withColumn("salted_key", F.concat_ws("_", "customer_id", "salt"))
```

25. When would you use salting?

Use salting when a join or aggregation has a highly skewed key and a small number of keys account for a large portion of the records. It is a technique for distributing hot keys more evenly.

26. What is the disadvantage of salting?

Salting adds complexity because the join or aggregation logic must account for the salt value. For joins, the smaller side may need to be expanded across the salt values. Therefore, salting should be used when skew is actually causing a bottleneck, not automatically for every join.

27. How is salting different from repartitioning?

Repartitioning changes how data is distributed across partitions. Salting changes the key used for distribution so that records belonging to a heavily repeated key can be spread across multiple partitions. Salting specifically targets key skew.

4. Delta Lake & Databricks

28. What is VACUUM in Delta Lake?

VACUUM removes old data files that are no longer referenced by the current Delta table state and are older than the configured retention period. It helps clean up storage, but retention settings should be handled carefully because old files may be needed for time travel or recovery scenarios.

```
VACUUM my_table RETAIN 168 HOURS
```

29. What is the difference between OPTIMIZE and VACUUM?

OPTIMIZE reorganizes data files, commonly by compacting small files to improve read performance. VACUUM removes obsolete files that are no longer needed according to the retention rules. In short: OPTIMIZE improves file layout; VACUUM cleans up old files.

30. What is the small-file problem?

The small-file problem occurs when a table contains a very large number of small data files. This can increase file-listing overhead and create many tasks with little work. Compaction, appropriate write strategies, and table optimization can help.

31. What is Delta Lake?

Delta Lake is a storage layer that adds features such as ACID transactions, schema enforcement, schema evolution options, time travel, and reliable updates/deletes to data stored in a data lake.

5. Spark Architecture

32. Explain the basic Spark architecture.

A Spark application typically has a Driver and one or more Executors. The Driver creates the Spark application context, builds the execution plan, and coordinates work. Executors run tasks and store data for the application. A cluster manager such as YARN, Kubernetes, or Spark Standalone can allocate resources.

33. What is the role of the Spark Driver?

The Driver is the control process for a Spark application. It creates the SparkSession/SparkContext, builds jobs from transformations and actions, creates stages and tasks, and coordinates execution with executors.

34. What is the role of an Executor?

Executors run tasks sent by the Driver and provide storage for cached or persisted data. Executors are usually long-lived for the duration of a Spark application.

35. What are jobs, stages, and tasks?

An action creates a Spark job. A job is divided into stages based on shuffle boundaries. Each stage is divided into tasks, usually one task per partition for that stage.

36. What is a shuffle in Spark?

A shuffle is the redistribution of data across partitions, commonly required by operations such as groupBy, join, distinct, and repartition. Shuffles can involve disk and network I/O, making them an important performance consideration.

37. What is the difference between SparkSession and SparkContext?

SparkSession is the modern high-level entry point for DataFrame and SQL operations. SparkContext is the lower-level entry point to Spark's core functionality and cluster communication. A SparkSession internally provides access to the SparkContext.

38. How does a Spark application execute an action?

When an action is called, Spark builds the logical plan, optimizes it, creates a physical plan, divides work into stages around shuffle boundaries, creates tasks for partitions, and schedules those tasks on executors.

39. What is a cluster manager?

A cluster manager allocates compute resources for Spark applications. Common choices include Spark Standalone, YARN, and Kubernetes. The cluster manager is responsible for resource allocation; Spark handles the application-level execution.

40. What is the difference between client mode and cluster mode?

In client mode, the Driver runs on the machine where the application was submitted. In cluster mode, the Driver runs inside the cluster. The choice affects where the Driver process and its network dependencies are located.

41. What happens when a Spark executor fails?

Spark can detect executor loss and reschedule unfinished tasks on other available executors, depending on the cluster configuration and available resources. Spark's lineage and recomputation mechanisms also help recover lost intermediate data.

42. Explain a typical PySpark data pipeline architecture.

A common architecture is: source systems → ingestion → cloud storage/data lake → PySpark transformation layer → curated tables such as Delta/Parquet → serving layer or warehouse → BI/reporting. In production, orchestration, monitoring, data quality, security, and catalog/governance are also important.

Key areas to practice: Spark execution model, transformations and actions, partitioning, shuffle, joins, AQE, data skew and salting, caching, Delta Lake, and Driver/Executor architecture.